

FIAP

PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE · 2º SEMESTRE

Revisão expressa + LangChain LCEL e ChatOllama

Aula 01/14 — Módulo 1: LangChain Foundations

 Prof. Jorge Luiz Gomes

 1h40min

 LCEL · ChatOllama · OutputParser

 Andaime 40%

Agenda da Aula

- 1** Revisão expressa do 1º semestre — mapa de conceitos em 20 min 20 min
- 2** Por que LangChain — o problema de orquestrar manualmente em produção 15 min
- 3** LCEL — o operador | como composição de pipeline 15 min
- 4** 🚀 Demo ao vivo — reescrever um chatbot manual em 15 linhas LCEL 20 min
- 5** 📄 Lab — Notebook Aluno: chain básica com domínio do grupo (40% lacunas) 20 min
- 6** Síntese + preview Aula 02 + CKP01 10 min

🎯 O QUE VOCÊ VAI CONSEGUIR FAZER AO FINAL DESTA AULA

Construir uma chain LangChain com LCEL que substitui o `chamar_llm()` manual do 1º semestre — com menos código, mais composição e pronto para escalar com memória e RAG nas próximas aulas.

Glossário — termos novos de hoje

Conceitos que você vai usar em **todas** as aulas deste semestre. Definição técnica direta — nada de decoreba:

LangChain */læŋtʃeɪn/*

Framework Python para orquestrar LLMs: conecta modelo, prompt, memória, RAG e tools em *chains* (cadeias) composáveis, em vez de código manual repetido.

LCEL

LangChain Expression Language — linguagem declarativa que compõe componentes com o operador `|` (pipe). A chain descreve o fluxo, não o código que o executa.

Runnable */ˈrʌnəbəl/*

Interface padrão que toda chain herda: `.invoke()`, `.stream()`, `.batch()`, `.ainvoke()`. É o "contrato" único para executar qualquer pipeline.

ChatPromptTemplate

Template de prompt com roles (system/human/ai) e variáveis `{var}` validadas em runtime — evolução da função `montar_prompt()` do 1º semestre.

ChatOllama

Integração oficial do LangChain com o Ollama (local ou Cloud): conecta o modelo (`qwen3.6:27b`) como componente de chat dentro de uma chain.

OutputParser

Componente que transforma a resposta bruta do modelo (AIMessage) em um formato útil para o seu código: texto, dict, ou objeto Pydantic validado.

StrOutputParser

Parser mais simples: extrai apenas o conteúdo textual da resposta. Elimina o `resp["message"]` `["content"]` do 1º semestre.

JsonOutputParser

Parser que converte a resposta em dict Python. Exige que o prompt peça JSON explicitamente — sem schema garantido (isso vem na Aula 03 com Pydantic).

chain

Pipeline de componentes encadeados pelo operador `|`. O output de um componente é o input do próximo.

stateless

Sem estado: cada chamada `.invoke()` começa do zero, sem lembrar conversas anteriores. A chain de hoje é stateless; memória é a Aula 02.

streaming

Receber a resposta token por token, em tempo real — efeito "digitando" em interfaces. Método `.stream()`, usado em produção e na Aula 08 com Gradio.

AIMessage

Objeto de resposta do LangChain: contém o texto em `.content`, além de metadados (uso de tokens, modelo, timestamps).

POR QUE ISSO IMPORTA NO MERCADO

LangChain é o framework de orquestração de LLMs mais usado em produção — a base de pipelines de geração de código, escrita assistida e automação de workflows em empresas de tecnologia. Domine o vocabulário de hoje (*chain*, *parser*, *Runnable*) e você fala a mesma língua das vagas de **AI Engineer** em 2026.

Mapa do 1º semestre — de onde você vem

10 aulas, 3 módulos — o que foi de fato aplicado nesta turma no 1º semestre (Módulo 1 ao Módulo 3, até a Aula 10). Nada disso será reexplicado hoje — mas você precisa reconhecer onde cada peça se encaixa antes de acelerar com LangChain.



Módulo 1 — Fundamentos de IA (Aulas 01–04)

O que é Inteligência Artificial e suas subáreas (Machine Learning, Deep Learning, NLP, Visão Computacional). Como os LLMs funcionam por dentro (tokens, embeddings, atenção). Ambiente Python configurado para IA.



Módulo 2 — Prompt Engineering Básico (Aulas 05–08)

Conceito e anatomia de um prompt. Técnicas zero-shot, few-shot e Chain-of-Thought. Role prompting e parâmetros do modelo (temperatura, top-p). Prompts para tarefas específicas.



Módulo 3 — Prompt Engineering Avançado (Aulas 09–10)

Avaliação e iteração de prompts com structured output. Prompt injection, jailbreak e guardrails — a base de segurança que volta na próxima seção.



10 aulas concluídas



3 módulos



Python + prompting já consolidados



Módulos 4–6 (LangChain, RAG, agentes, ética/carreiras) não aplicados — viram conteúdo novo deste 2º semestre



O QUE VOLTA HOJE SEM REEXPLICAÇÃO

Token · temperatura · top-p · janela de contexto · prompt injection · jailbreak · guardrail · structured output. Toda chain que você construir com LangChain herda os mesmos riscos do 1º semestre — um `ChatPromptTemplate` mal desenhado é tão vulnerável a prompt injection quanto o system prompt manual. A defesa em camadas continua sendo sua responsabilidade; Context Engineering chega com mais profundidade nas Aulas 04 e 10 deste semestre.

Segurança em LLMs — revisão: prompt injection, jailbreak e guardrails

Este bloco recapitula a Aula 10 do 1º semestre (Módulo 3 — Prompt Engineering Avançado: "Prompt Injection, Segurança e Guardrails"). Não é conteúdo novo — é o que garante que a chain LCEL que você vai montar hoje não repita as mesmas vulnerabilidades do system prompt manual do 1º semestre.

⚠️ ATAQUES QUE VOCÊ JÁ VIU

- Prompt injection direto** — o usuário tenta sobrescrever o system prompt na própria mensagem ("ignore as instruções anteriores...").
- Prompt injection indireto** — instrução maliciosa embutida num dado processado pelo modelo (e-mail, PDF, página web sendo resumida).
- Prompt leaking** — extrair o system prompt oculto através de perguntas indiretas.
- Jailbreaking** — roleplay, encoding (Base64) ou personas fictícias para contornar guardrails.

🔒 POR QUE ISSO NUNCA DESAPARECE

LLMs não distinguem instrução do desenvolvedor de instrução do usuário — tudo é texto na mesma *context window* (janela de contexto). Isso vale igual para o `ChatPromptTemplate` que você vai construir hoje.

🛡️ DEFESA EM CAMADAS — AS 5 QUE VOCÊ JÁ APRENDEU

- ✓ **System prompt robusto** — regras invioláveis + sandwich defense (regras repetidas no início e no fim)
- ✓ **Validação de input** — bloquear padrões como "ignore", "esqueça", "system prompt"
- ✓ **Separação dados↔instruções** — delimitadores (crases triplas, tags XML)
- ✓ **Validação de output** — checar dado sensível vazado antes de responder
- ✓ **Monitoramento e logging** — kill switch para respostas fora do esperado

🔗 CONEXÃO COM A AULA 04 E A AULA 10 DESTES SEMESTRE

Context Engineering (Anthropic, 2025) trata o contexto como recurso finito — quanto mais informação irrelevante no contexto, menos atenção o modelo dá aos guardrails. A Aula 04 aprofunda essa curadoria; a Aula 10 aplica defesa em camadas a agentes com tools.

RANK	RISCO OWASP LLM TOP 10 (2025)	ONDE É TRATADO NO CURSO
#1	Prompt Injection	1º sem · Aula 10 (Módulo 3) · reforçado hoje
#2	Insecure Output Handling	Mitigado pelo OutputParser de hoje (slide 13)
#6	Sensitive Information Disclosure	Mencionado no 1º sem (Módulos 2–3) · LGPD
#8	Excessive Agency	2º sem · Aula 10 deste semestre (agentes com tools)
#9	Overreliance	Mencionado no 1º sem (Módulos 2–3)

Técnicas de prompt e Python que você já escreve

Nenhuma dessas habilidades desaparece com LangChain — elas migram para dentro dos componentes da chain.

✓ TÉCNICAS DE PROMPT

Zero-shot · Few-shot · Chain-of-Thought
Role prompting · System prompt com persona
Parâmetros: temperatura, top-p, num_predict

✓ PYTHON QUE VOCÊ SABE FAZER

def, while True/break, f-string multiline, **dict, retorno múltiplo, continue

Onde essas técnicas vão morar em LCEL

- ✓ Few-shot e CoT viram exemplos dentro do ChatPromptTemplate
- ✓ Role prompting vira a mensagem system do template
- ✓ Parâmetros (temperatura, top-p) viram argumentos do ChatOllama(...)
- ✓ def montar_prompt() vira um ChatPromptTemplate.from_messages([...])
- ✓ while True do loop de chat vira .stream() ou memória gerenciada

💡 NADA FOI PERDIDO

Você não está aprendendo prompting do zero — está aprendendo a **declarar** em LCEL o que já sabia escrever manualmente em Python puro.

O ponto de partida: o chatbot manual do 1º semestre

Este é o código que você já escreveu e que serve de base para tudo que vem hoje. Guarde-o na cabeça — o próximo bloco reescreve exatamente isto em LCEL.

PYTHON — CHATBOT MANUAL DO 1º SEMESTRE (~40 LINHAS, RESUMIDO)

```
def montar_prompt(histórico, pergunta):
    return [{"role": "system", "content": SYSTEM_PROMPT}] \
        + histórico + [{"role": "user", "content": pergunta}]

def chamar_llm(messages):
    resp = ollama.chat(model="qwen3.6:27b", messages=messages)
    return resp["message"]["content"]

def iniciar_chat():
    histórico = []
    while True:
        pergunta = input("Você: ")
        if pergunta == "sair": break
        msgs = montar_prompt(histórico, pergunta)
        resposta = chamar_llm(msgs)
        histórico += [{"role": "user", "content": pergunta},
                      {"role": "assistant", "content": resposta}]
    print("Bot:", resposta)
```

✅ Chatbot manual — o que ele já faz

chamar_llm() → iniciar_chat() → montar_prompt() · lista messages com 3 roles · histórico gerenciado manualmente em cada volta do loop.

🔗 O PONTO DE PARTIDA DESTA AULA

Três funções, gerenciamento manual de `histórico`, acesso direto a `resp["message"]["content"]`. Funciona — mas não escala, não compõe e não tem memória além do que você programar à mão. Esse chatbot manual (~40 linhas) é o problema que o LangChain resolve: hoje você reescreve isso em 15 linhas usando LCEL.

01

Por que LangChain

O problema de escalar orquestração manual — e como o framework resolve com composição declarativa.

O problema do 1º semestre em produção

O chatbot manual funciona para um usuário num Colab. Mas o que acontece quando você precisa escalar?

CENÁRIO	COM O CÓDIGO MANUAL	COM LANGCHAIN
Trocar o modelo	Reescrever a chamada à API em cada função	<code>ChatOllama(model="outro")</code> — 1 linha
Adicionar memória	Gerenciar a lista <code>histórico</code> manualmente	Trocar tipo de memória em 1 parâmetro
Parsear o output	<code>resp["message"]["content"]</code> em todo lugar	OutputParser declarado uma vez na chain
Compor chains	Chamar cada função manualmente em sequência	<code>chain1 chain2 chain3</code>
Adicionar RAG	Refatorar toda a arquitetura do chatbot	Inserir retriever na chain existente
Rastreamento/debug	Print statements espalhados	LangSmith integrado automaticamente

Esforço relativo de manutenção (1 = trivial · 10 = refatoração completa)

Cada cenário do slide anterior, quantificado: quanto custa mudar com código manual vs. com uma chain LCEL



Passe o mouse (ou toque) sobre os elementos do gráfico para ver os detalhes.

LEITURA DO GRÁFICO

Esses valores representam o **esforço de mudança** (1–10) para 6 operações comuns. Com LCEL, cada mudança vira uma linha de código: `ChatOllama(model="outro")` , trocar o parser, inserir um retriever. O gráfico reflete por que o framework existe: **o custo marginal de evoluir cai de "refatorar" para "1 linha"**.

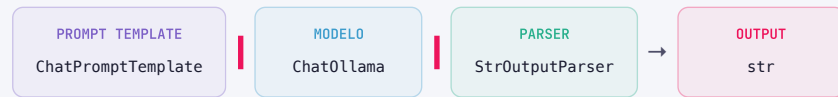
02

LCEL — LangChain Expression Language

O operador `|` como composição. Uma chain é a declaração de como os dados fluem — não o código que os move.

LCEL — o operador | como pipe

O LCEL usa o operador `|` (pipe) para compor componentes. O output de um vira input do próximo:



```
LCEL_BÁSICO.PY

from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Cada componente é independente – compostos pelo operador |

# 1. Template: define estrutura e variáveis do prompt
prompt = ChatPromptTemplate.from_messages([
    ("system", "Você é {persona}. Responda sobre {especialidade}."),
    ("human", "{pergunta}"),
])

# 2. Modelo: conecta ao Ollama Cloud
llm = ChatOllama(
    model="qwen3.6:27b",
    base_url="https://ollama.com",
)

# 3. Parser: extrai só o texto da resposta
parser = StrOutputParser()

# Composição com | – isso é uma Runnable, não uma chamada
chain = prompt | llm | parser

# Invocar a chain com as variáveis do template
resposta = chain.invoke({
    "persona": "um chef de culinária brasileira",
    "especialidade": "culinária brasileira",
    "pergunta": "Como faço um bolo de cenoura?",
})

print(resposta) # → string direta, sem resp["message"]["content"]
```

ChatPromptTemplate — evolução do `montar_prompt()`

`ChatPromptTemplate` substitui a função `montar_prompt()` que você escreveu no 1º semestre — com validação automática de variáveis:

← BACK 1º SEMESTRE — F-STRING MANUAL

```
def montar_prompt(pergunta,
                  especialidade,
                  tom="direto"):
    return f"""
Você é especialista em {especialidade}.
Pergunta: {pergunta}
Tom: {tom}.
"""

# Problema: f-string não valida variáveis
# se esquecer uma, silencia sem erro
```

🧑 2º SEMESTRE — CHATPROMPTTEMPLATE

```
prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Você é especialista em {especialidade}."
     " Tom: {tom}."),
    ("human", "Pergunta: {pergunta}"),
])

# Vantagem: valida variáveis automaticamente
# KeyError se esquecer alguma ao invocar
# Separação de roles limpa (system/human/ai)
```

💡 FROM_MESSAGES VS. FROM_TEMPLATE

`from_messages([("system", "..."), ("human", "...")])` — quando você precisa de roles diferentes (a maioria dos casos com LLMs de chat).

`from_template("texto simples com {var}")` — para prompts de texto puro, sem roles.

Output Parsers — StrOutputParser e JsonOutputParser

O parser define como o output do modelo é transformado antes de chegar ao seu código:

STROUTPUTPARSER — TEXTO LIMPO

```
from langchain_core.output_parsers import StrOutputParser

chain = prompt | llm | StrOutputParser()

# Retorna string direta
resposta = chain.invoke({"pergunta": "0i"})
print(type(resposta)) # → <class 'str'>

# Sem mais resp["message"]["content"] !
```

JSONOUTPUTPARSER — DICT DIRETO

```
from langchain_core.output_parsers import JsonOutputParser

chain = prompt | llm | JsonOutputParser()

# O prompt DEVE pedir JSON explicitamente
resultado = chain.invoke({
    "pergunta": "Liste 3 ingredientes"
})
print(type(resultado)) # → <class 'dict'>
print(resultado["ingredientes"])
```

⚠ JSONOUTPUTPARSER SEM SCHEMA = RISCO

O `JsonOutputParser` tenta parsear qualquer JSON que o modelo gere — mas não garante que as chaves esperadas existam. Na Aula 03 você vai usar **PydanticOutputParser**, que valida o schema e levanta erro se a saída não bater.

ChatOllama — conectando ao Ollama Cloud

No 1º semestre você usou `ollama.Client(host=..., headers={...})`. No LangChain, o setup é declarativo:

SETUP_OLLAMA_CLOUD.PY

```
!pip install langchain-ollama langchain-core -q

from langchain_ollama import ChatOllama
from google.colab import userdata
import os

# Definir a API key via variável de ambiente (Colab Secrets)
os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")

# Instanciar o modelo — parâmetros declarativos
llm = ChatOllama(
    model="qwen3.6:27b",          # modelo padrão do semestre
    temperature=0.7,             # mesmo parâmetro do 1º sem
    num_predict=512,             # equivalente a max_tokens
)

# Teste rápido direto (sem chain)
resp = llm.invoke("Olá! Responda em 1 linha.")
print(resp.content)  # .content — não ["message"]["content"]
print(type(resp))    # → AIMessage (objeto LangChain)
```

✅ DIFERENÇA CHAVE: RESPOSTA DIRETA VS. CHAIN

`llm.invoke("texto")` → retorna um `AIMessage` — precisa de `.content` para acessar o texto.

`chain.invoke({...})` com `StrOutputParser` → retorna `str` diretamente. Use chains na maioria dos casos.

Runnable — o que toda chain herda

Qualquer componente LCEL implementa a interface `Runnable`. Isso dá 4 métodos gratuitos:

`.invoke()`

Chama a chain com 1 input e retorna 1 output. O método mais usado.

```
chain.invoke({"pergunta": "..."})
```

`.stream()`

Retorna generator — tokens chegam um a um. Usado em interfaces para efeito "digitando".

```
for chunk in chain.stream({...}): print(chunk, end="")
```

`.batch()`

Processa lista de inputs em paralelo. Útil para avaliar prompts com múltiplos casos de teste.

```
chain.batch([{"q": "A"}, {"q": "B"}])
```

`.ainvoke()`

Versão assíncrona de invoke. Usado em servidores web (FastAPI, Gradio) para não bloquear.

```
await chain.ainvoke({...})
```

REGRA PRÁTICA PARA O SEMESTRE

Use `.invoke()` em notebooks e testes. Use `.stream()` na Aula 08 quando você adicionar interface Gradio. Use `.batch()` para avaliar seu RAG na Aula 07.

04

Lab — Notebook Aluno

40% de lacunas — componentes da chain e chamada final com o domínio que o grupo vai usar no semestre inteiro.

Notebook Aluno — 40% de lacunas

Complete as lacunas. Esta chain é a base de todo o semestre — o domínio escolhido hoje será usado nos CKPs 01, 02 e 03.

```
!pip install langchain-ollama langchain-core -q

from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from google.colab import userdata
import os

os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")

# 🚩 LACUNA 1: instancie o ChatOllama com o modelo certo
llm = ChatOllama(model=____, temperature=____)

# 🚩 LACUNA 2: crie o ChatPromptTemplate com 2 roles
# system: persona + especialidade do grupo
# human: a pergunta do usuário
prompt = ChatPromptTemplate.from_messages([
    (____, ____), # system com persona do grupo
    (____, ____), # human com {pergunta}
])

# 🚩 LACUNA 3: componha a chain com os 3 componentes
chain = ____ | ____ | ____

# 🚩 LACUNA 4: invoque com uma pergunta do domínio do grupo
resposta = chain.invoke({"pergunta": ____})
print(resposta)

# Bônus: testar .stream() – observe os tokens chegando
for chunk in chain.stream({"pergunta": "Me dê um exemplo rápido."}):
    print(chunk, end="", flush=True)
```

AULA_01_2SEM_ALUNO.IPYNB

Como esta chain cresce ao longo do semestre

A chain de hoje é a base. Cada aula adiciona uma peça:

A1 Hoje — prompt | llm | StrOutputParser() · chain stateless básica

A2 Aula 02 — adicionar memória: ConversationChain + BufferMemory · chain stateful

A3 Aula 03 — adicionar validação: PydanticOutputParser · saída com schema garantido

A5 Aula 05–07 — adicionar RAG: retriever | prompt | llm | parser · contexto de documentos

A9 Aula 09 — adicionar tools: chain → agente com ferramentas · RAG como tool do agente

A13 Aula 13 — LangGraph: grafo de estado com routing condicional · além do loop linear

FUNDAMENTAÇÃO — PROMPT CHAINING

Gullí (*Agentic Design Patterns*, 2025) descreve **Prompt Chaining** como o padrão fundacional de todo sistema agêntico: decompor uma tarefa complexa em etapas menores, cada uma processada por um componente especializado, com o output de uma etapa alimentando a próxima. A chain LCEL de hoje — `prompt | llm | parser` — é a versão mais simples desse padrão. Os agentes das Aulas 09–10 aplicam a mesma ideia, com decisões condicionais entre os elos.

CKP01 — preview do que o grupo vai construir

🚩 CKP01 — CHATBOT LANGCHAIN · GRUPO 3-4 · ENTREGA AULA 04

Reescrever o chatbot manual do 1º semestre usando LangChain. **Requisito mínimo:** ConversationChain com 1 tipo de memória + schema Pydantic validando a saída + system prompt com persona + domínio real documentado.

✅ **R1:** ChatOllama + ChatPromptTemplate + StrOutputParser

✅ **R2:** ConversationChain com 1 tipo de memória

✅ **R3:** PydanticOutputParser com schema definido

✅ **R4:** Domínio documentado + 5 turnos de conversa

Rubrica: 30% chain LCEL · 25% memória · 25% Pydantic · 20% código/comentários

💡 DECISÃO DE HOJE IMPORTA PARA O SEMESTRE INTEIRO

O domínio escolhido na Aula 01 será mantido pelo semestre: CKP01 (chatbot), CKP02 (RAG sobre documentos do domínio), CKP03 (agente com tools do domínio). Escolha um domínio rico em documentos disponíveis publicamente — você vai precisar de PDFs na Aula 06.

LangChain no mercado — padrões de produção com LCEL

Os mesmos 3 componentes de hoje (prompt · modelo · parser) aparecem, com variações, em categorias inteiras de produtos de IA:

ASSISTENTES DE CÓDIGO

LangChain orquestrando múltiplos LLMs

Ferramentas de código assistido orquestram chains de geração e revisão — o modelo gerador e o modelo revisor são componentes diferentes numa mesma pipeline LCEL. Você vai construir algo similar na Aula 12 com LangGraph.

ESCRITA ASSISTIDA

ChatPromptTemplate por contexto

Templates de prompt diferentes por tipo de tarefa (resumo, expansão, brainstorm) — cada um com variáveis que recebem o conteúdo do documento atual. É exatamente o padrão que você aprendeu hoje com ChatPromptTemplate.

AUTOMAÇÃO DE WORKFLOWS

Chains como nós de workflow

Plataformas de automação (RPA + IA) usam LangChain por baixo — cada ação automatizada é uma chain que recebe contexto do usuário, chama o LLM e retorna output estruturado para o próximo passo do workflow. OutputParser com Pydantic garante que os dados chegam no formato certo.

AUTOMAÇÃO DE PROCESSOS (BPM)

Triagem e resposta padronizada

Plataformas de BPM com IA usam LangChain para automatizar triagem de tickets e geração de respostas padronizadas — ChatPromptTemplate por tipo de solicitação, JsonOutputParser para routing automático.

Erros comuns ao começar com LangChain

ERRO	CAUSA	SOLUÇÃO
<code>ModuleNotFoundError: langchain_ollama</code>	Pacote separado — LangChain não vem com a integração Ollama por padrão	<code>!pip install langchain-ollama langchain-core -q</code>
<code>KeyError: 'pergunta'</code> ao invocar	A variável no template usa <code>{pergunta}</code> mas o dict passou <code>"question"</code>	Nomes devem bater exatamente — case-sensitive
Resposta vem como <code>AIMessage</code> não como <code>str</code>	Esqueceu o <code>StrOutputParser()</code> no final da chain	Sempre terminar a chain com <code> StrOutputParser()</code>
JsonOutputParser falha com <code>JSONDecodeError</code>	O modelo não gerou JSON válido — prompt não pediu explicitamente	Incluir no template: <i>"Responda SOMENTE em JSON válido."</i>
Autenticação falha (<code>401 Unauthorized</code>)	<code>os.environ["OLLAMA_API_KEY"]</code> não definido antes de instanciar o modelo	Definir env vars antes do <code>ChatOllama(...)</code>

Síntese da Aula 01



LangChain não faz nada que você não poderia fazer manualmente — o valor é a composição declarativa. `prompt` | `llm` | `parser` descreve o fluxo; o framework cuida da orquestração.



ChatPromptTemplate substitui `montar_prompt()` com validação automática de variáveis e separação limpa de roles (`system/human/ai`).



StrOutputParser e JsonOutputParser eliminam o acesso manual a `resp["message"]["content"]` — o output já chega no tipo certo.



Toda chain herda Runnable — `.invoke()`, `.stream()`, `.batch()`, `.ainvoke()` sem código adicional.



O domínio escolhido hoje é o fio condutor do semestre — CKP01 (chatbot), CKP02 (RAG), CKP03 (agente) todos usam o mesmo domínio.

Perguntas frequentes

PERGUNTA	RESPOSTA
"Preciso desinstalar o pacote ollama do 1º semestre?"	Não — eles coexistem sem conflito. <code>ollama</code> (cliente original) e <code>langchain-ollama</code> (integração LangChain) são pacotes diferentes. Você pode ter os dois instalados simultaneamente.
"O <code> </code> é o operador OR do Python?"	Em Python puro, <code> </code> é OR bitwise e union de dicts (Python 3.9+). No LangChain, a classe <code>Runnable</code> sobrescreve o método <code>__or__</code> para retornar uma chain composta em vez de OR.
"Por que usar ChatPromptTemplate se f-string também funciona?"	Para projetos simples, f-string funciona. A vantagem do template aparece na escala: validação de variáveis em tempo de execução, reutilização com valores diferentes, integração automática com chains mais complexas (memória, RAG, agentes).
"Preciso de conta no LangSmith para usar LangChain?"	Não — LangSmith é a plataforma de rastreamento (opcional). Todo o código do semestre roda sem LangSmith. Se quiser ativar, é gratuito para uso pessoal e adiciona observabilidade automática.
"A versão do LangChain importa?"	Muito — o LangChain teve mudanças de API significativas entre versões. O semestre usa LangChain 0.3+ (LCEL estável). Sempre instale com <code>pip install langchain-core langchain-ollama</code> para pegar a versão atual.
"Posso usar outros modelos além de qwen3.6:27b?"	Sim — o semestre usa qwen3.6:27b como padrão, mas qualquer modelo disponível no Ollama Cloud funciona. Outros modelos como llama4:scout ou deepseek-r2 também funcionam bem, dependendo da tarefa.
"LCEL vai mudar? Vale aprender agora?"	LCEL foi estabilizado na versão 0.3 e é a API de longo prazo do LangChain. O padrão <code>prompt llm parser</code> é o que a documentação oficial recomenda e o que toda a comunidade adotou.

Resumo — o que é novo no Python desta aula

CONCEITOS_PYTHON_AULA01_2SEM.PY

1. Instância declarativa com keyword arguments

```
llm = ChatOllama(model="qwen3.6:27b", temperature=0.7)
```

2. Método de classe com lista de tuplas

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "Você é {persona}."),
    ("human", "{pergunta}"),
])
```

3. Operator overloading – | cria chain composta

```
chain = prompt | llm | StrOutputParser()
```

4. .invoke() – método com dict de variáveis

```
resp = chain.invoke({"persona": "chef", "pergunta": "Olá"})
```

5. Generator com for + yield (em .stream())

```
for chunk in chain.stream({"pergunta": "Oi"}):
    print(chunk, end="", flush=True)
# flush=True força exibição imediata de cada token
```

6. os.environ – variáveis de ambiente em Python

```
import os
os.environ["OLLAMA_API_KEY"] = "valor" # dict especial do SO
```


O semestre em 14 aulas — onde você vai chegar

M1 — LANGCHAIN

Aulas 01–04
LCEL · Memória
Pydantic
Context Eng.
CKP01

M2 — RAG

Aulas 05–08
Embeddings
ChromaDB
Gradio
CKP02

M3 — AGENTES

Aulas 09–11
ReAct · Tools
RAG como tool
URL pública
CKP03

M4 — LANGGRAPH

Aulas 12–14
Router · Grafo
HITL
Tendências

🎯 AO FINAL DO SEMESTRE VOCÊ VAI TER

Um agente com interface Gradio pública que usa RAG sobre documentos do seu domínio + busca web + executa código Python — orquestrado com LangGraph. Isso é o CKP03. O CKP01 (hoje) é o primeiro passo.

03

Demo ao vivo

Reescrever um chatbot manual de 40 linhas para 15 linhas LCEL — mesmo comportamento, menos código.



Demo Prática ao Vivo

O professor abre um notebook com o chatbot manual do 1º semestre e o notebook LCEL de hoje lado a lado no Colab, executa os dois ao vivo com a mesma pergunta e compara o resultado linha a linha.



Dois notebooks lado a lado

Chatbot manual (1º semestre) à esquerda, chain LCEL de hoje à direita — mesma pessoa, mesma pergunta de teste.



Execução ao vivo

Os dois códigos rodam no Colab em tempo real — a turma observa a resposta chegar igual nos dois casos.



Comparação linha a linha

O professor destaca onde cada função manual (montar_prompt, iniciar_chat, parsing do dict) foi substituída por um componente da chain.

40→15

linhas de código

3→1

funções manuais → 1 chain

0

acessos a resp["message"]
["content"]

STACK USADO NA DEMO

Google Colab · Ollama Cloud API · qwen3.6:27b · langchain-core + langchain-ollama



LAB — AULA 01 · 2º SEMESTRE

Construir a chain base do grupo ★★

Grupo 3-4 · 20 minutos · Google Colab

1. **Escolha o domínio do grupo** — este domínio vai persistir pelo semestre inteiro: CKP01 (chatbot LangChain), CKP02 (RAG sobre documentos desse domínio), CKP03 (agente). Exemplos: culinária, direito do consumidor, saúde, games, finanças pessoais.
2. **Complete as 4 lacunas** — modelo, template com persona do domínio, composição com | e chamada com `.invoke()`.
3. **Teste com 3 perguntas do domínio** — confirme que a persona está coerente com o domínio escolhido.
4. **Desafio — JsonOutputParser**: crie uma segunda chain com `JsonOutputParser`. No prompt, peça ao modelo para responder em JSON com as chaves resposta e confiança (0-10). Imprima o tipo do resultado.

🎯 Orientação das lacunas

Lacuna 1: nome do modelo do semestre e a temperatura usada desde o 1º semestre (0.7).

Lacuna 2: a role de sistema recebe a persona e o domínio do grupo; a role humana recebe a variável da pergunta.

Lacuna 3: os 3 componentes na ordem prompt → modelo → parser, unidos pelo operador pipe.

Lacuna 4: qualquer pergunta real do domínio escolhido pelo grupo.

💡 **Dica para o desafio:** no prompt, adicione: "Responda SOMENTE em JSON no formato: {\"resposta\": \"...\", \"confiança\": N}". O modelo precisa da instrução explícita para gerar JSON válido.

Referências

DOCS **LangChain** — LCEL (LangChain Expression Language): composição declarativa de chains com o operador |. python.langchain.com/docs/concepts/lcel

DOCS **LangChain** — ChatPromptTemplate: estruturar prompts com roles e variáveis. python.langchain.com/docs/concepts/prompt_templates

DOCS **LangChain** — Output parsers: StrOutputParser, JsonOutputParser, PydanticOutputParser. python.langchain.com/docs/concepts/output_parsers

DOCS **langchain-ollama** — ChatOllama: integração LangChain com modelos Ollama. python.langchain.com/docs/integrations/chat/ollama

LIVRO **Russell, S.; Norvig, P.** — Inteligência Artificial. 3ª ed. Pearson, 2016. Cap. 2: Agentes racionais — o modelo de percepção → ação que fundamenta o conceito de pipeline de LLM.

EBOOK **Polzer, D.** — RAG with Python Cookbook. O'Reilly, 2025. Cap. 1: escolha de frameworks para aplicações RAG e por que LangChain (LCEL) é o padrão de mercado para orquestração.

EBOOK **Gullí, A.** — Agentic Design Patterns. O'Reilly, 2025. Cap. 1: Prompt Chaining — a base dos padrões agênticos; o LCEL que você aprendeu hoje é a fundação dos agentes da Aula 10.

EBOOK **Lanham, M.** — AI Agents in Action. Manning, 2025. Cap. 2: prompting LLMs com personas e delimitadores — reforça o que você construiu no 1º semestre.

Aula 01 concluída 🗑️🗑️

O que levou 40 linhas manuais no 1º semestre agora são 3 linhas de composição. Na Aula 02 você adiciona memória — e entende quando trocar de tipo.

→ PRÓXIMA AULA — AULA 02 · 10/08

Memória conversacional — Buffer, Summary e TokenBuffer

A chain de hoje é **stateless** — cada `.invoke()` começa do zero, igual à lista histórico manual do 1º semestre, que crescia sem limite de tokens. A Aula 02 resolve isso com 3 tipos de memória — Buffer, Summary e TokenBuffer — e o CKP01 fica mais próximo.

📌 Decisão de hoje

Anote o domínio escolhido pelo grupo. Ele acompanha o semestre inteiro:
CKP01 → CKP02 → CKP03.